PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No. 9 (A)

Aim : To implement any code optimization techniques

Objective: Develop a program to implement following code optimization techniques:

a. Common sub-expression elimination
b. Reduction in strength

Outcome: Students are able to appreciate role of code optimization and implement various techniques.

Theory:

Compilers should produce target code that is as good as a hand written code. This is quiet difficult in reality. The code generated by compiler can be made to run faster or take less space. This improvement is achieved through some program transformations which are called as optimizations.
Compilers that apply code-improving transformations are called optimizing compilers.

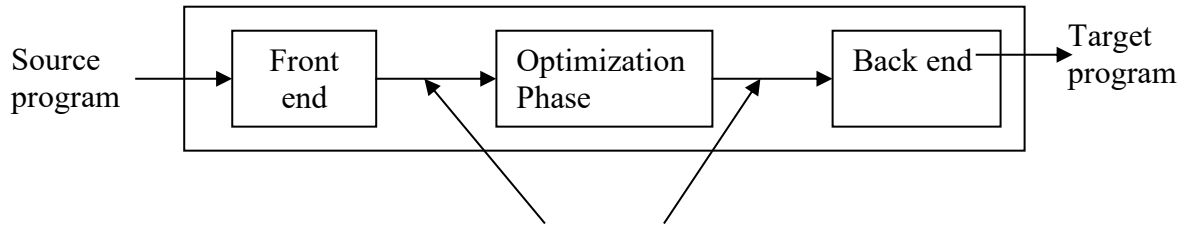


Fig : Schematic of optimizing compiler

A compiler can improve a program without changing the function it computes, using function preserving transformations. The different function-preserving transformations are:

1. Common sub-expressions elimination
2. Copy propagation
3. Dead-code elimination and
4. Constant folding

- **Common sub-expressions**

  If an expression E was previously computed and the values of variables in E have not changed since previous computation, an expression E is called as common sub-expression. We can use previously computed value to avoid re-computation of the expression.
  Consider the following code segment in basic block B1

t6 := 4 * i	**t6 := 4 * i**
x := a[t6]	**x := a[t6]**
t7 := 4 * i	**t8 := 4 * j**
t8 := 4 * j	**t9 :=a[t8]**
t9 :=a[t8]	**a[t6] := t9**
a[t7] := t9	**a[t8] := x**
t10 := 4 * j	**goto B2**
a[t10] := x	
goto B2	

(a) Before (b) **After**

FIGURE Local common sub-expression elimination

- **Reduction in strength**

x := x * 2 à x := x + x

à x := x << 2

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No.: **C-54**	Name: **Quazi Md Moinuddin**
Class: **TE-C**	Batch: **C-3**
Date of Experiment:**26.03.2025**	Date of Submission: **15.04.2025**
Grade:	

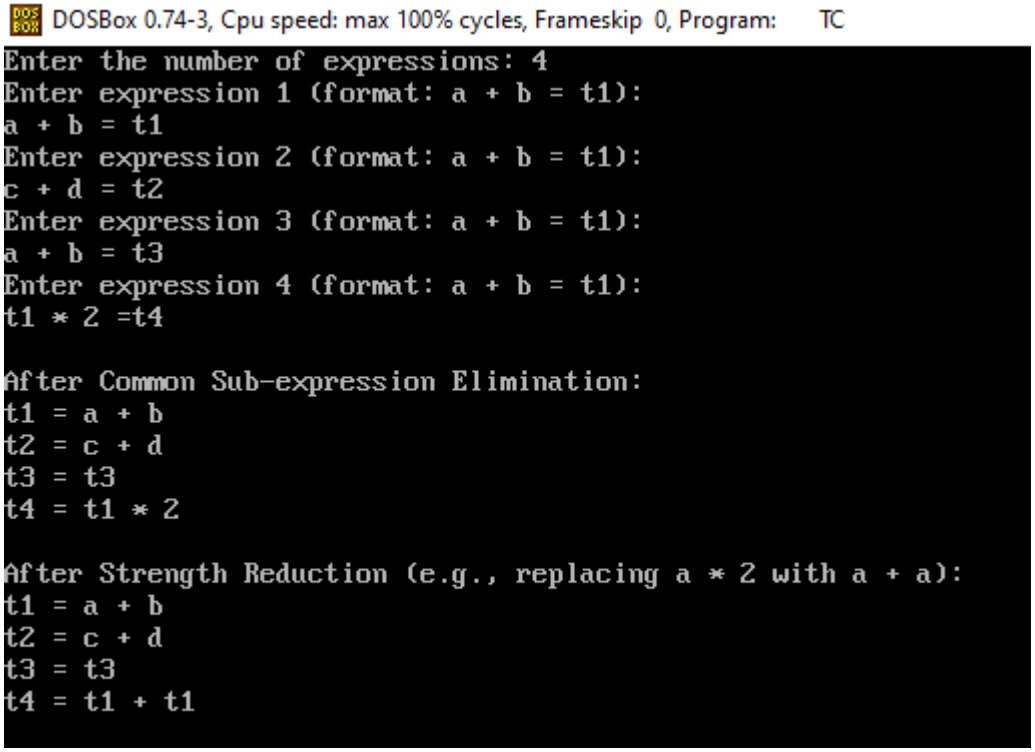
B.1 Software Code written by student:

(Paste your code completed during the 2 hours of practical in the lab here)

```
#include <stdio.h>
#include <string.h>
#include <conio.h>

struct expr {
   char op1[10];
   char op2[10];
   char operator;
   char result[10];
};

void main() {
   struct expr exp[10];
   int i, j, n, ch;
   char temp[10];
   clrscr();

   do {
      printf("Enter the number of expressions: ");
      scanf("%d", &n);

      for (i = 0; i < n; i++) {
         printf("Enter expression %d (format: a + b = t1):\n", i + 1);
         scanf("%s %c %s = %s", exp[i].op1, &exp[i].operator, exp[i].op2, exp[i].result);
      }

      printf("\nAfter Common Sub-expression Elimination:\n");
```

```
      for (i = 0; i < n; i++) {
         for (j = i + 1; j < n; j++) {
            if ((strcmp(exp[i].op1, exp[j].op1) == 0 && strcmp(exp[i].op2, exp[j].op2) == 0
&&
                 exp[i].operator == exp[j].operator) ||
                (strcmp(exp[i].op1, exp[j].op2) == 0 && strcmp(exp[i].op2, exp[j].op1) == 0
&&
                 exp[i].operator == exp[j].operator)) {
                strcpy(temp, exp[j].result);
                strcpy(exp[j].op1, temp);
                strcpy(exp[j].op2, "");
                exp[j].operator = '=';
            }
         }
      }

      for (i = 0; i < n; i++) {
         if (exp[i].operator == '=') {
            printf("%s = %s\n", exp[i].result, exp[i].op1);
         } else {
            printf("%s = %s %c %s\n", exp[i].result, exp[i].op1, exp[i].operator, exp[i].op2);
         }
      }

      printf("\nAfter Strength Reduction (e.g., replacing a * 2 with a + a):\n");

      for (i = 0; i < n; i++) {
         if (exp[i].operator == '*' && strcmp(exp[i].op2, "2") == 0) {
            printf("%s = %s + %s\n", exp[i].result, exp[i].op1, exp[i].op1);
         } else {
            if (exp[i].operator == '=') {
               printf("%s = %s\n", exp[i].result, exp[i].op1);
            } else {
               printf("%s = %s %c %s\n", exp[i].result, exp[i].op1, exp[i].operator, exp[i].op2);
            }
         }
      }

      printf("\nDo you want to continue? (1 for Yes / 0 for No): ");
      scanf("%d", &ch);
      printf("\n-----------------------------------------\n");

   } while (ch == 1);

   printf("\nPress any key to exit...");
   getch();
}
```

B.2 Input and Output:



B.3 Observations and learning:

- ❖ Understood how repeated expressions can be optimized using common sub-expression elimination.
- ❖ Learned how expensive operations (like multiplication) can be replaced with cheaper alternatives (like addition) through strength reduction.
- ❖ Explored the role of optimization in reducing execution time and memory usage in compiled programs.

B.4 Conclusion:

Code optimization techniques like Common Sub-expression Elimination and Strength Reduction help improve the efficiency of code without changing its meaning. These optimizations are widely used in compilers and play a crucial role in speeding up execution and saving resources.

B.5 Question of Curiosity

1. Comment on live at that point ?

A variable is said to be live at a point in a program if its current value may be used in the future. It helps the compiler decide whether to keep the value in memory/register or eliminate it to save space.

2. What are the types of block transformation?

Block transformations involve optimizing basic blocks in a program. Types include:

1. Dead Code Elimination – Removing code that doesn't affect the program result.
2. Constant Folding – Precomputing constant expressions.
3. Common Sub-expression Elimination – Avoiding repeated evaluation.
4. Code Motion – Moving code outside loops.
5. Strength Reduction – Replacing expensive operations with cheaper ones.

3. State the Normal form of Block.

The Normal Form of a Basic Block refers to a simplified and optimized version where:

- Each expression appears only once.
- Redundant code and dead statements are removed.
- Variables are used only if live.
- Expressions are reordered if necessary, without changing the meaning.